

Отчет по лабораторной работе

Макаренко Матвей 932320. Лабораторная 1.

Диспетчер Умного Дома (SmartHome Hub)

Проблема:

Проблема заключается в том, что создание конфигурации устройств «Умного дома» требует обеспечения строгой совместимости компонентов внутри одной экосистемы. При этом протоколы связи, формат данных и функционал кардинально отличаются в зависимости от производителя (например, премиум-экосистема требует шифрования и расширенной телеметрии, а бюджетная — базового управления). Использование простого условного кода для создания устройств приведет к нарушению принципа открытости/закрытости (ОСР), усложнению кода при добавлении новых вендоров и риску несовместимости устройств разных производителей в рамках одной сессии управления.

Пример из кода (без паттерна):

```
// ✗ Нарушение ОСР: для нового вендора нужно менять этот класс
if (currentVendor == "EcoHome")
{
    ecoLight = new EcoLight();
    ecoThermostat = new EcoThermostat();
}
else if (currentVendor == "TechPro")
{
    techLight = new TechLight();
    techThermostat = new TechThermostat();
}
```

Решение:

Применён паттерн Abstract Factory для создания семейств связанных объектов (устройств одного вендора) без привязки к конкретным классам.

Компоненты решения:

DeviceFactory — абстрактная фабрика, объявляющая методы создания продуктов (CreateLight(), CreateThermostat(), CreateLock()). EcoHomeFactory и TechProFactory — конкретные фабрики для создания устройств бюджетной и премиум экосистем соответственно. Light, Thermostat, Lock — интерфейсы продуктов, определяющие общие методы для всех устройств. Конкретные классы (EcoLight, TechLight и др.) реализуют

функционал для конкретного вендора. Клиентом в данной архитектуре выступает MainForm — GUI-форма, которая работает исключительно с интерфейсами фабрик и продуктов, не зная о конкретных классах устройств.

Пример из кода (с паттерном):

// Соблюдение ОСР: для нового вендора создаётся новый класс фабрики

```
private void SetFactory(DeviceFactory factory)
```

```
{
```

```
    _factory = factory;
```

```
    _light = factory.CreateLight();
```

```
    _thermostat = factory.CreateThermostat();
```

```
    _lock = factory.CreateLock();
```

```
}
```

// Вызов из GUI:

```
SetFactory(new EcoHomeFactory()); // или new TechProFactory()
```

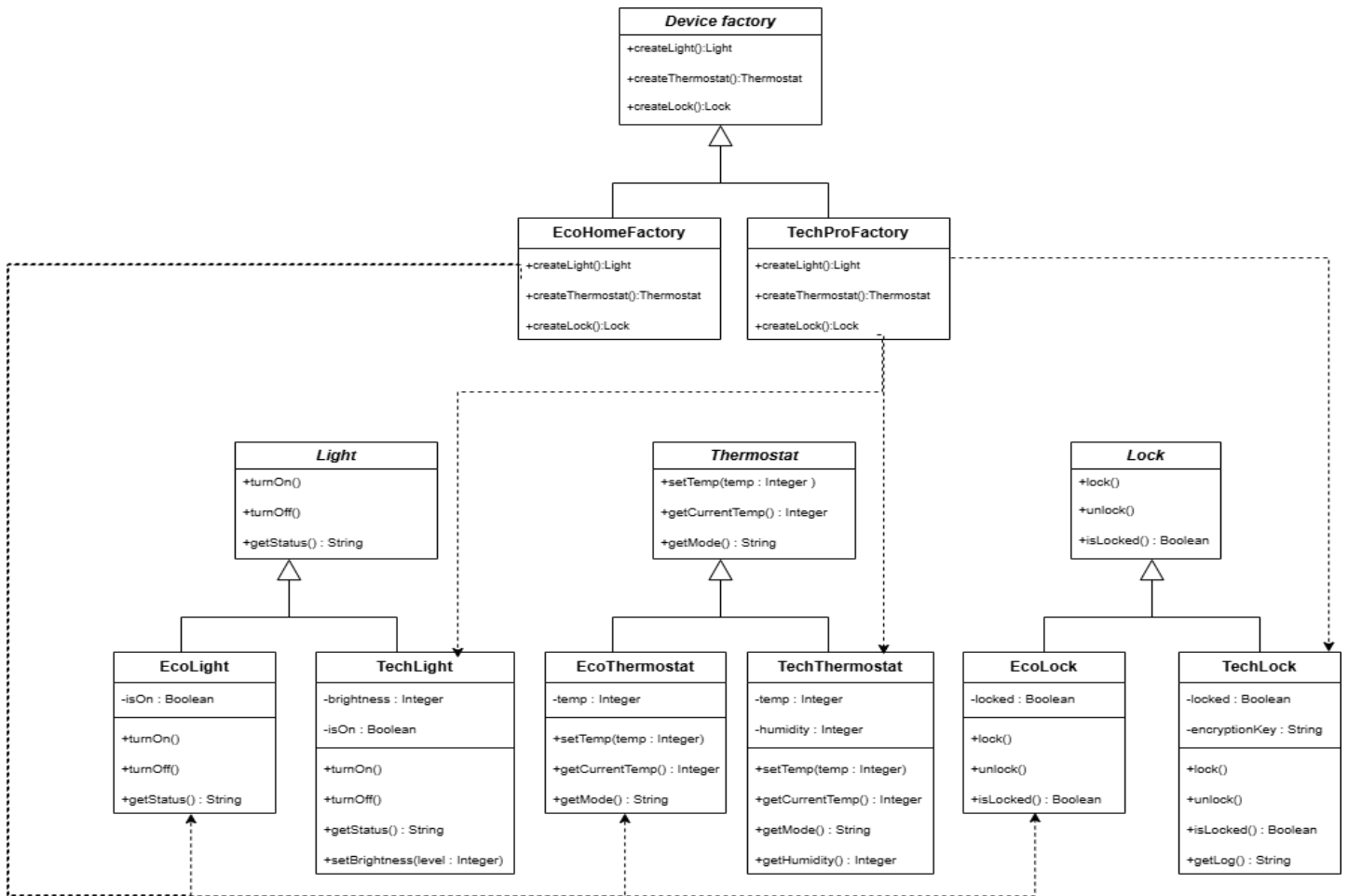


Рисунок 1 - Диаграмма классов паттерна Abstract Factory в архитектуре приложения SmartHome Hub

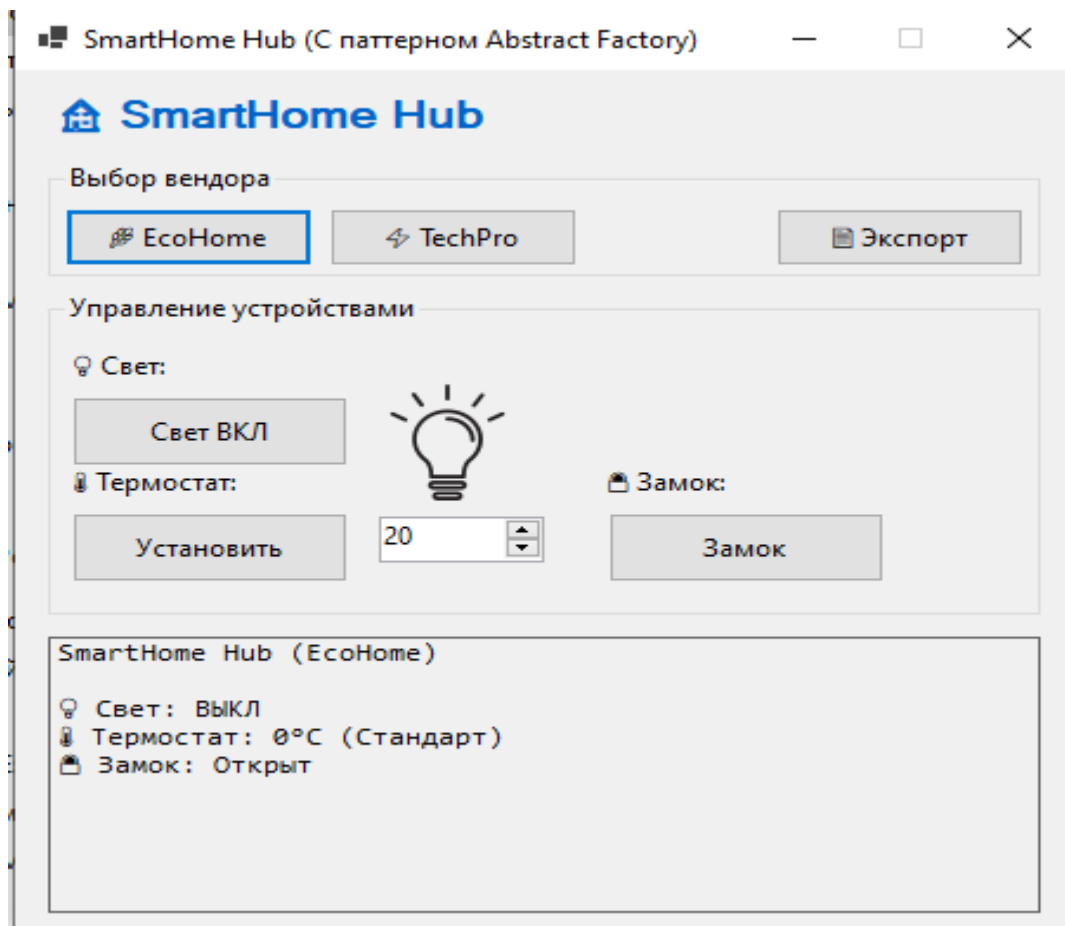


Рисунок 2 - Интерфейс приложения SmartHome Hub (режим EcoHome)

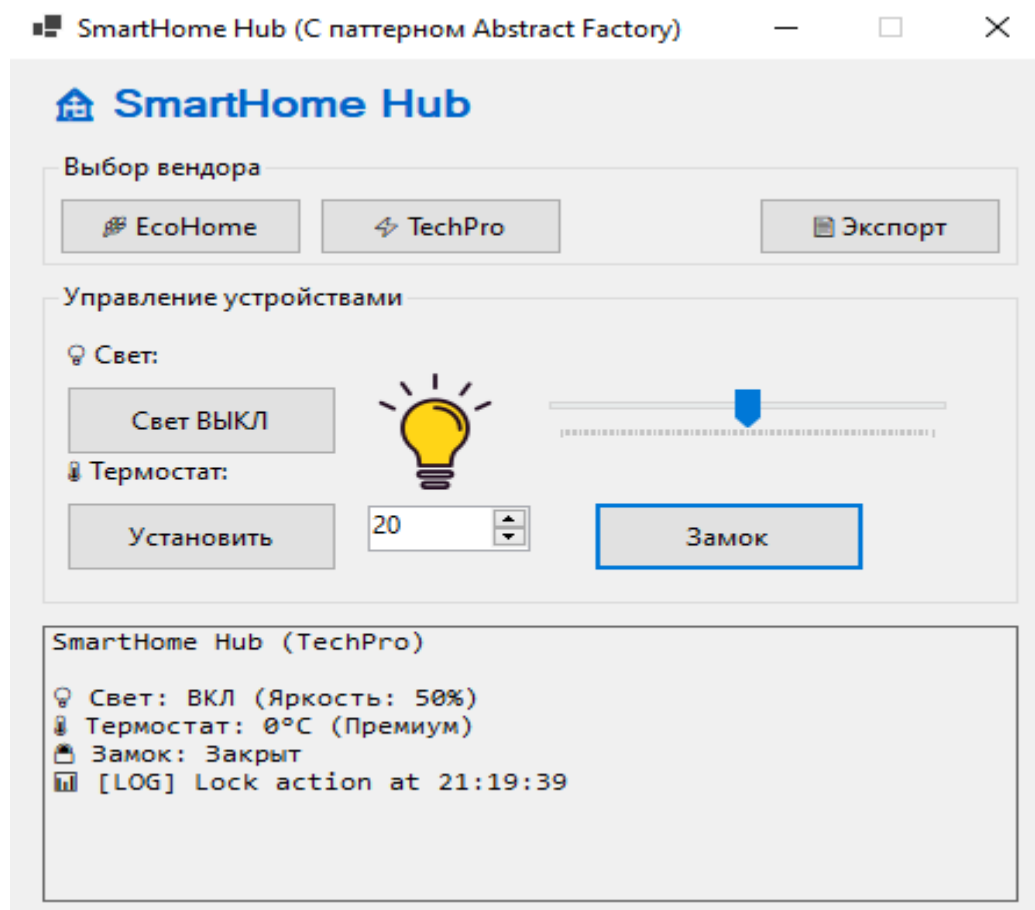


Рисунок 3 - Интерфейс приложения SmartHome Hub (режим TechPro)

Вывод:

Применение паттерна Abstract Factory позволило устранить нарушение принципа Open/Closed, снизить связность между компонентами системы и гарантировать совместимость устройств внутри одной экосистемы. Для добавления нового вендора достаточно создать 4 новых класса (3 продукта + 1 фабрика), не изменяя существующий код клиента (MainForm). Это делает систему расширяемой, поддерживаемой и устойчивой к ошибкам конфигурации.